

Building a Simple Single-Cycle RISC Datapath

AArch64-style datapath construction using one-cycle execution and a worked `ldr` instruction path

Dr. J. Burns

Topic 8 Outline

1. Single-Cycle Microarchitecture

- Architectural state, combinational logic, and one-cycle execution.

2. Datapath State Elements

- PC, instruction memory, register file, ALU, immediate generation, data memory.

3. Worked Load Path

- Step-by-step execution of `ldr x7, [x5, #8]`.

4. Extending the Datapath

- Store, data-processing, branch paths, and the control signals that select them.

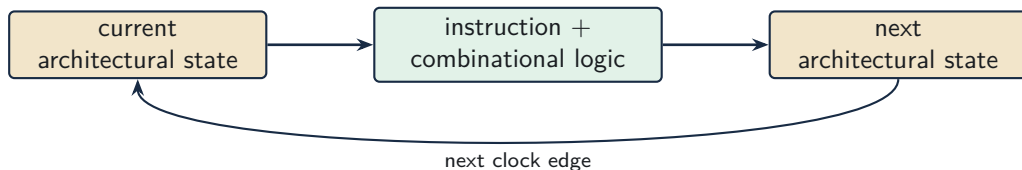
PART 1

Single-Cycle Microarchitecture

Microarchitecture

- The **ISA** defines the instructions visible to software.
- The **microarchitecture** is the hardware organization that executes those instructions.
- A single-cycle datapath is one microarchitectural implementation of the AArch64 ISA.
- The datapath moves values through memories, registers, ALUs, and multiplexers.
- The control unit interprets the instruction and selects the datapath actions.
- Later implementations can use pipelining while preserving the same ISA behavior.

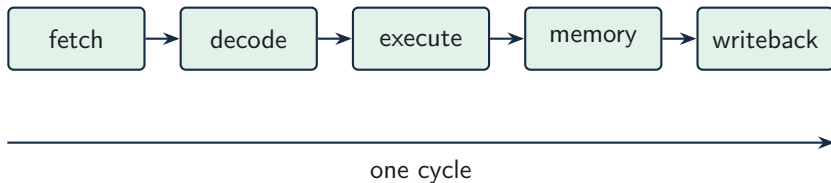
Architectural State as a Finite-State Machine



- State elements hold the visible machine state between cycles.
- Combinational logic computes the next state from the current state and instruction.
- A clock edge commits the new state.

Single-Cycle Execution

- One instruction completes in one clock cycle.
- Fetch, decode, execute, memory access, and writeback all fit in that cycle.
- State is updated at the clock edge at the end of the cycle.
- The clock period must be long enough for the slowest supported instruction.



AArch64 State Used Here

- **PC**: address of the current instruction.
- **Register file**: 32 architectural register numbers, 5 bits each.
- **NZCV**: condition flags used by conditional branches.
- The PC is modeled as a separate register, not as x15 or r15.
- x31 is interpreted as sp or xzr, depending on the instruction form.
- This simplified datapath treats the register file as two read ports and one write port.

AArch64 differs from the older ARM32 teaching model: there is no general-purpose r15 program counter register.

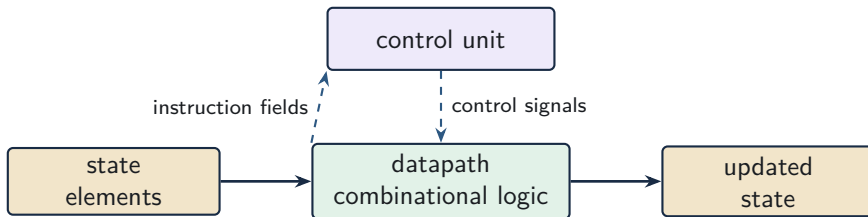
Instruction Subset

- **Memory:** `ldr xT, [xN, #imm]` and `str xT, [xN, #imm]` using unsigned scaled offsets.
- **Data processing:** `add`, `sub`, `and`, `orr` in register and simple immediate forms.
- **Control flow:** `b label` and simple conditional branches such as `b.eq label`.
- The load instruction is traced first because it exercises fetch, register read, immediate generation, ALU address calculation, data memory, and writeback.

This is not a full AArch64 implementation. It is a small datapath that is large enough to show the hardware pattern.

State Elements and Core Datapath Blocks

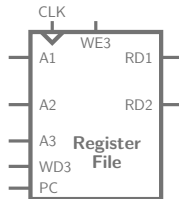
Datapath and Control



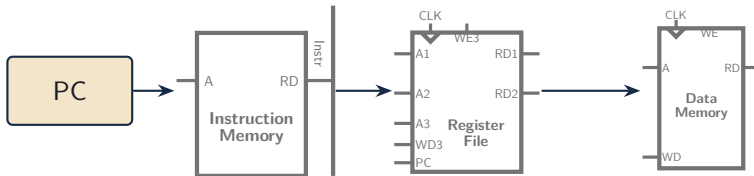
- The datapath contains the values and arithmetic.
- The control unit chooses which path the current instruction uses.

Register File

- A1 and A2 select the two registers to read.
- A3 selects the register to write.
- WD3 carries the write data.
- WE3 enables the write at the clock edge.
- RD1 and RD2 carry the two read values.

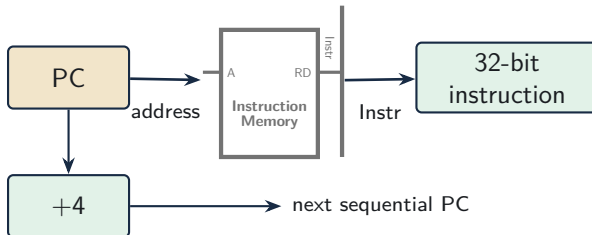


State Elements



- The PC supplies the address for instruction fetch.
- The register file stores integer register values.
- Data memory is read by `ldr` and written by `str`.
- Writes to architectural state happen at the end of the cycle.

Program Counter and Instruction Memory



AArch64 instructions are 4 bytes, so the sequential PC value is $PC + 4$.

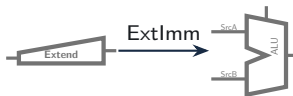
ALU and Immediate Generator

ALU

- computes add/subtract/logical results;
- produces address calculations for loads and stores;
- can produce NZCV flags for flag-setting forms.

Immediate Generator

- extracts immediate fields from the instruction;
- scales load/store offsets when required;
- sign-extends branch offsets.



AArch64 Immediate Modes in This Datapath

Instruction form	Immediate field	Datapath value
ldr/str xT, [xN, #imm]	imm12	zero-extended and scaled by access size
add/sub xD, xN, #imm	imm12	zero-extended immediate
b label	imm26	sign-extended, shifted left by 2
b.cond label	imm19	sign-extended, shifted left by 2

The control unit selects the immediate mode using an ImmSrc signal.

Data Memory

- The ALU computes the effective address from a base register and offset.
- `ldr` reads data memory and writes the loaded value to the register file.
- `str` reads a register value and writes that value to data memory.
- The simplified model treats memory as a separate instruction memory and data memory.
- The same address-generation path is used by both `ldr` and `str`.

PART 3

Worked Load Path

Load Example

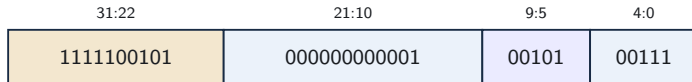
```
ldr x7, [x5, #8]
```

- Read the base address from x5.
- Add the byte offset 8 to form the effective address.
- Read 8 bytes from data memory.
- Write the loaded 64-bit value into x7.
- Update the PC to the next sequential instruction at the end of the cycle.

For a 64-bit AArch64 load with unsigned offset, the encoded `imm12` is $8/8 = 1$.

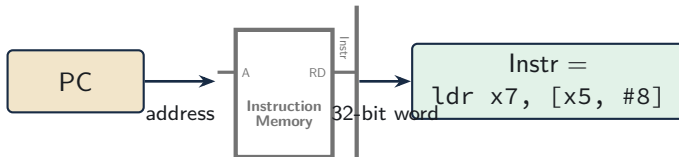
Load Instruction Fields

`ldr x7, [x5, #8]` is encoded as `0xF94004A7`.



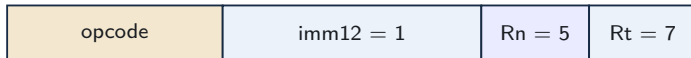
Field	Value	Meaning
opcode/form	1111100101	64-bit load, unsigned immediate
imm12	000000000001	scaled offset 1, meaning 8 bytes
Rn	00101	base register x5
Rt	00111	destination register x7

Step 1: Fetch the Instruction



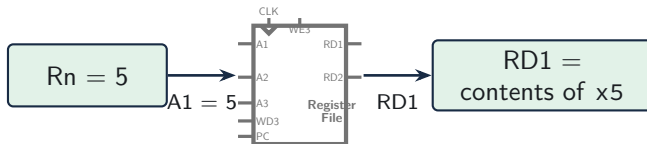
The PC supplies the instruction address. Instruction memory returns the 32-bit AArch64 instruction word.

Step 2: Decode Register Fields



- Rn selects the base register: x5.
- Rt selects the destination register: x7.
- The register file reads x5 while the rest of the datapath computes the offset and address.

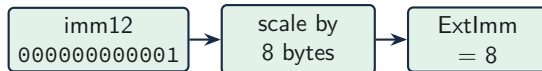
Step 3: Read the Base Register



The base address is not embedded in the instruction. The instruction names the register that holds the base address.

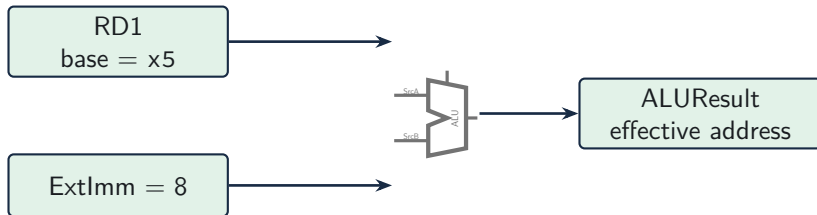
Step 4: Generate the Offset

- Field value: $\text{imm12} = 1$.
- Access size: 8 bytes, because this is `ldr x`.
- Byte offset: $1 \times 8 = 8$.



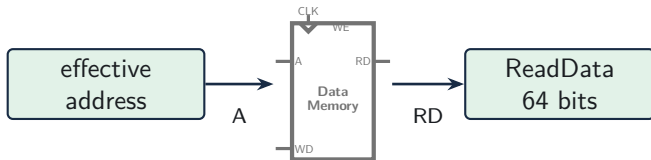
AArch64 unsigned load/store offsets are scaled by the access size.

Step 5: Compute the Effective Address



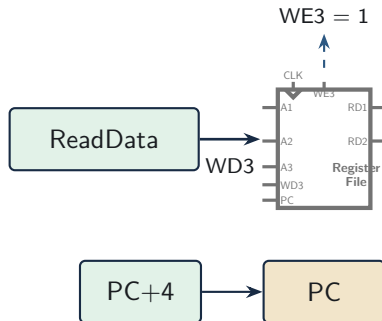
For a load or store, the ALU is used as an address adder.

Step 6: Read Data Memory



The memory read value becomes the candidate writeback value for x7.

Step 7: Write Back and Update PC



At the clock edge, x7 receives the loaded value and the PC receives $PC + 4$.

Load Path Control Settings

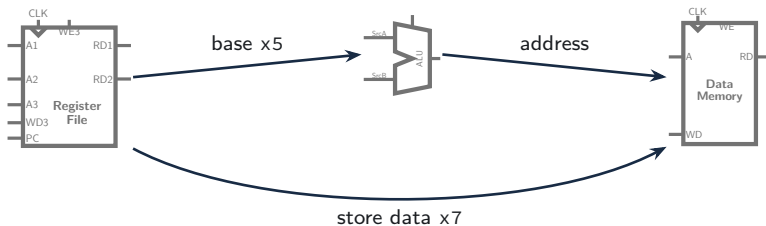
Signal	Setting for <code>ldr x7, [x5, #8]</code>	Effect
RegWrite	1	write loaded value to x7
MemWrite	0	do not write data memory
MemToReg	1	writeback comes from data memory
ALUSrc	1	ALU second input is the extended offset
ALUControl	add	compute $\text{base} + \text{offset}$
ImmSrc	load/store scaled	produce byte offset 8
PCSrc	0	next PC is $PC + 4$

These control values select one path through the same hardware.

Extending the Datapath

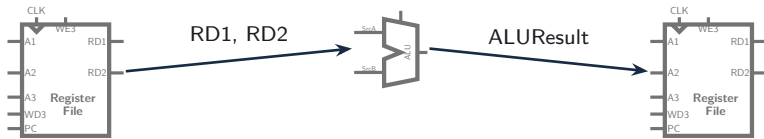
Store Path: `str x7, [x5, #8]`

`str x7, [x5, #8]` uses the same address calculation as the load.



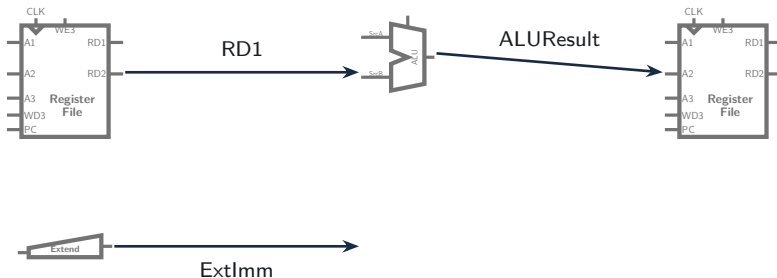
- $Rt = x7$ is read from the register file as store data.
- $MemWrite = 1$ and $RegWrite = 0$.

Register ALU Path: `add x1, x2, x3`



- No data memory access is needed.
- `MemToReg = 0`, so the writeback value comes from the ALU.
- `ALUSrc = 0`, so the ALU second operand comes from the register file.

Immediate ALU Path: add x1, x2, #3



For immediate ALU instructions, `ALUSrc = 1` selects the immediate generator output.

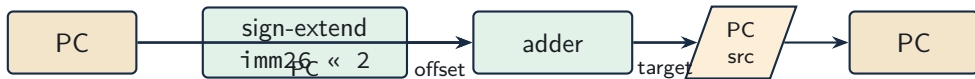
Second Register Source Selection

AArch64 register fields are not all used the same way across instruction families.

Instruction form	Register file read port 1	Register file read port 2
<code>add xD, xN, xM</code>	Rn bits 9:5	Rm bits 20:16
<code>add xD, xN, #imm</code>	Rn bits 9:5	not used
<code>ldr xT, [xN, #imm]</code>	Rn bits 9:5	not used
<code>str xT, [xN, #imm]</code>	Rn bits 9:5	Rt bits 4:0

A small multiplexer selects whether the second read address comes from Rm or Rt.

Branch Path: b label



A branch replaces $PC + 4$ with a PC-relative target address.

Control Signals and Timing

Control Signals

Signal	Meaning
RegWrite	enable a write to the register file
MemWrite	enable a write to data memory
MemToReg	choose memory data or ALU result for writeback
ALUSrc	choose register data or immediate as ALU input B
ALUControl	select add, subtract, and, or other ALU operation
ImmSrc	select the immediate-extension mode
Read2Src	choose Rm or Rt for the second register read
PCSrc	choose sequential PC or branch target

Example Control Settings

Instruction	RegW	MemW	MemToReg	ALUSrc	ALU	ImmSrc	Read2Src	PCSrc
ldr xT, [xN, #imm]	1	0	1	1	add	load/store	—	0
str xT, [xN, #imm]	0	1	X	1	add	load/store	Rt	0
add xD, xN, xM	1	0	0	0	add	—	Rm	0
sub xD, xN, xM	1	0	0	0	sub	—	Rm	0
add xD, xN, #imm	1	0	0	1	add	add/sub imm	—	0
b label	0	0	X	X	add	branch	—	1

The entries marked X are not used by that instruction.

Single-Cycle Limitations

- Hardware is easy to reason about, but timing is inefficient.
- Every instruction waits for the worst-case datapath delay.
- Simple instructions cannot finish early.
- Some hardware must be duplicated when two values are needed in the same cycle.
- The next topic reuses the same datapath ideas but overlaps instructions in a pipeline.

PART 6

Summary and Review

Topic 8: Summary

- A single-cycle datapath computes the next architectural state in one clock period.
- State elements store the PC, registers, memory contents, and flags.
- The `ldr x7, [x5, #8]` path uses instruction fetch, register read, immediate scaling, ALU address calculation, memory read, and register writeback.
- The same datapath is extended with multiplexers and control signals for stores, ALU instructions, and branches.

Self-Test Questions: Datapath Basics

- ❶ In a single-cycle datapath, when are state elements updated?
- ❷ Why is the PC modeled separately from the AArch64 general-purpose register file?
- ❸ For `ldr x7, [x5, #8]`, which register supplies the base address?
- ❹ Why is the encoded `imm12` equal to 1 rather than 8 for that instruction?
- ❺ What is the role of the ALU in a load or store instruction?

Self-Test Questions: Control and Timing

- ❶ Which control signal distinguishes a register ALU operand from an immediate ALU operand?
- ❷ Why does `str` assert `MemWrite` but not `RegWrite`?
- ❸ What value does `MemToReg` select for a load?
- ❹ Why does a single-cycle implementation have a long clock period?
- ❺ Which datapath components are used by `add x1, x2, x3` but not by `b label`?

Looking Ahead

- The single-cycle datapath shows the full instruction path in one cycle.
- Pipelining splits that path into stages so several instructions can be in progress at once.
- The next topic uses the same fetch, decode, execute, memory, and writeback structure to build the 5-stage RISC pipeline.
- Hazards appear when overlapped instructions need the same data or control decision.